ЛАБОРАТОРНА РОБОТА № 2
Паралельна обробка наборів даних із використанням multiprocessing

Метою лабораторної роботи є набуття практичних навичок використання багатопроцесорної обробки для аналізу табличних наборів даних та експериментального дослідження ефективності паралельного виконання обчислень.

Для лабораторної роботи рекомендується використовувати:

```
from multiprocessing import Pool
```

`Pool` створює групу робочих процесів, між якими можна розподіляти незалежні задачі.

Набір даних Online Retail II

Завдання 1. Завантаження наборів даних

Завдання 2. Первинний аналіз

Для кожного набору визначити:

1. кількість рядків;
2. кількість стовпців;
3. назви стовпців;
4. типи даних;
5. кількість пропущених значень;
6. кількість дубльованих рядків.

Завдання 3. Підготовка даних

Перевірити пропущені значення в:

- `Description`;
- `CustomerID`;
- інших полях.

Також необхідно:

- перетворити `InvoiceDate` у формат дати;
- перевірити `Quantity`;
- перевірити `UnitPrice`;
- визначити скасовані транзакції.

Завдання 4. Формування розрахункового показника

Необхідно сформувати **нову числову ознаку**, яку надалі буде використано для паралельної обробки.

```
Revenue = Quantity × UnitPrice
```

Завдання 5. Фільтрація даних

Для кожного набору виконати фільтрацію записів за створеним числовим показником.

Визначити записи, для яких значення відповідного показника:

x>mean(x)

Тобто необхідно знайти записи, значення яких перевищує середнє значення показника.

Використати:

`Revenue > середнє Revenue`

Визначити:

- кількість таких записів;
- їхню частку від загальної кількості;
- сумарне значення показника.

Завдання 6. Групування даних

Групування за:

`Country`

Для кожної країни визначити:

- кількість записів;
- сумарну виручку;
- середню виручку.

Завдання 7. Визначення Top-N

Визначити Top-10 країн за:

`Revenue`

Результат представити у вигляді таблиці.

Завдання 8. Реалізація функції обробки частини даних. Послідовна обробка

Необхідно створити універсальну функцію, яка отримує частину DataFrame і повертає часткові результати.

Наприклад:

```
def process_chunk(df, value_column, group_column):
    ...
```

Функція повинна визначати для переданої частини:

1. кількість записів;
2. суму;
3. середнє;
4. мінімальне значення;
5. максимальне значення;
6. групову статистику.

Виконати всю обробку **одним процесом** та визначити витрачений час.

Завдання 9. Розподіл DataFrame

Для паралельної обробки необхідно розділити DataFrame на декілька частин.

Наприклад:

```
import numpy as np

chunks = np.array_split(
    df,
    n_processes
)
```

Зроити паралелльно для 4х процесів.

Кожна частина повинна оброблятися незалежно.

Завдання 10. Паралельна обробка

Імпортувати:

```
from multiprocessing import Pool
```

Створити пул процесів:

```
with Pool(processes=n_processes) as pool:
    results = pool.starmap(
        process_chunk,
        ...
    )
```

Завдання 11. Об'єднання часткових результатів

Після завершення роботи процесів необхідно об'єднати отримані результати.

Особливу увагу приділити правильному об'єднанню:

- сум;
- кількостей;
- мінімальних значень;
- максимальних значень;
- середніх значень;
- групових результатів.

Завдання 12. Перевірка правильності

Порівняти:

```
результат послідовної обробки
            VS
результат паралельної обробки
```

Результати повинні збігатися з допустимою похибкою для операцій із числами з плаваючою комою.

Необхідно перевірити щонайменше:

- кількість записів;
- суму;
- середнє;
- мінімум;
- максимум;
- результати групування;
- Top-10.

У звіті зазначити:

Послідовний і паралельний алгоритми дають однаковий результат.

або, якщо результати відрізняються:

Вказати причину розбіжності та спосіб її усунення.

Завдання 13. Експеримент із кількістю процесів. Вимірювання часу. Розрахунок Speedup. Розрахунок Efficiency.

Провести експеримент із різною кількістю процесів:

- 1;
- 2;
- 4;
- 8.

Кількість процесів повинна бути адаптована до апаратних можливостей комп'ютера.

Кількість доступних логічних процесорів можна визначити:

```
import os

print(os.cpu_count())
```

Процеси	Час, с	Speedup	Efficiency
1		1,00	
2			
4			
8			

Завдання 14. Побудова графіків

1. Побудувати графік:

```
X — кількість процесів
Y — час виконання
```

2. Побудувати графік `Speedup`:

```
X — кількість процесів
Y — Speedup
```

3. Побудувати графік `Efficiency`:

```
X — кількість процесів
Y — Efficiency, %
```

Проаналізувати, як змінюється ефективність зі збільшенням кількості процесів.

Завдання 15. Визначення оптимальної кількості процесів

За результатами експерименту визначити:

Яка кількість процесів є найбільш ефективною для даної задачі та даного комп'ютера?

Врахувати:

- час виконання;
- Speedup;
- Efficiency;
- кількість доступних CPU;
- розмір набору даних.